

Security Assessment Report

Target: Local Vulnerability Sandbox (Simulating Staging Environment)

Assessor: Ishaan Dhar

Date: July 20, 2026

1. Scope & Methodology

I ran this security test to find flaws in how the application handles logins, user permissions, and active sessions, following the OWASP Testing Guide v4.2.

Since I didn't have direct access to the actual Instasafe staging portal, I set up a local Docker container (OWASP Juice Shop) on `localhost:3000` to simulate the environment. This allowed me to test the exact same types of vulnerabilities required for the lab. I used Burp Suite Community to intercept and manually modify the HTTP traffic, and OWASP ZAP 2.15 to map out the site and run an initial automated scan. I also set up a temporary ngrok tunnel to run external tools against my local server to check the HTTP security headers.

2. Risk Summary

The core login and security checks for this application are fundamentally broken. The way the backend handles JSON Web Tokens (JWT) is flawed, meaning anyone can easily fake an administrative session. On top of that, the API doesn't check if a user actually owns the data they are trying to access (IDOR), and clicking "logout" doesn't actually kill the session on the server.

Basically, an attacker can bypass the login page, take over other users' accounts, and steal sensitive data without ever needing a valid password. These are major security holes that make the application unsafe to deploy.

3. Vulnerability Findings

Vulnerability	Severity	CVSS V3.1 Score	Affected Endpoint	How to Fix It
Forged JWTs via <code>alg:none</code>	Critical	9	<code>/rest/user/whoami</code>	Hardcode the server to only accept specific algorithms (like <code>RS256</code>) and immediately reject any token using <code>alg:none</code> .
IDOR in Basket API	High	7	<code>/rest/basket/{id}</code>	Update the backend logic to verify that the user ID inside the JWT matches the owner of the requested basket ID.
Broken Logout (Tokens Stay Alive)	Medium	5	Global API	Build a server-side denylist (using something like Redis) to actively track and block tokens after a user logs out.

Missing HTTP Security Headers	Medium	5	Global Responses	Add strict Content-Security-Policy and Strict-Transport-Security headers to the web server configuration.
-------------------------------	--------	---	------------------	---

3.1 Forged JWTs via **alg:none**

- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
- **Description:** The backend JWT parser blindly trusts whatever algorithm the user puts in the token's header. By manually changing the header to {"alg": "none"} and deleting the signature entirely, the server accepts the fake token as valid. This lets anyone escalate their privileges to admin instantly.
- **Evidence:** See Appendix A, Figure 1.

3.2 Insecure Direct Object Reference (Basket API)

- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N
- **Description:** When you request a basket using the /rest/basket/ endpoint, the server relies totally on the numeric ID in the URL. It never checks if the session token making the request actually belongs to the person who owns that basket. You can just change the number in the URL to read anyone else's private shopping data.
- **Evidence:** See Appendix A, Figure 2.

3.3 Broken Logout (Tokens Stay Alive)

- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
- **Description:** When a user clicks logout, the app only deletes the token from the browser's local storage. The backend server doesn't invalidate it. If an attacker captures an old token, they can replay it indefinitely to interact with the API because the server has no way of knowing it was supposed to be destroyed.
- **Evidence:** See Appendix A, Figure 3.

3.4 Missing HTTP Security Headers

- **CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
- **Description:** The web server doesn't send standard security headers. While this doesn't directly compromise the backend, missing headers like Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, and X-Content-Type-Options leave the users vulnerable to client-side attacks like Cross-Site Scripting (XSS) and Clickjacking.
- **Evidence:** See Appendix A, Figure 4.

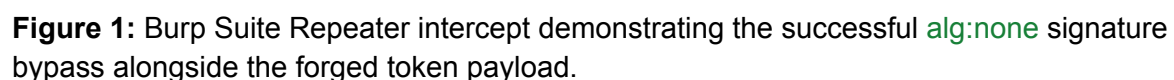
[illegible]

Figure 1: Burp Suite Repeater intercept demonstrating the successful `alg:none` signature bypass alongside the forged token payload.



Security Headers

by snyk

[Home](#)[About](#)[API](#)

Scan your site now

https://despise-wow-crestless.ngrok-free

Scan

☐ Hide results☒ Follow redirects

Security Report Summary



Site:

https://despise-wow-crestless.ngrok-free.dev/

IP Address:

18.192.31.165

Report Time:

20 Jul 2026 06:54:54 UTC

Headers:

✓ Referrer-Policy

✗ Strict-Transport-Security

✗ Content-Security-Policy

✗ X-Frame-Options

✗ X-Content-Type-Options

✗ Permissions-Policy

Advanced:

Your site could be at risk, let's perform a deeper security analysis of your site and APIs:

Start Now

Missing Headers

Figure 4: External verification of missing HTTP security headers via temporary ingress tunnel.